NAME : KISHORE KUMAR V

REG NO : 192521371

COURSE CODE : ITA 0613

Session 6: Naive Bayes Algorithm

Session Topic: Write a program to implement Naive Bayes algorithm in Python and to display the results using confusion matrix and accuracy.

Question 1: Naive Bayes from Scratch for Spam Mail Detection

Aim:

To implement the Naive Bayes algorithm from scratch in Python for spam detection and to display the results using a confusion matrix and accuracy.

Algorithm:

1. Start. Load the training data (presence of the words Free, Offer, Winner and the class Spam / Ham).
2. Calculate the prior probability P(class) of Spam and Ham.
3. Calculate the conditional probability P(word | class) with Laplace smoothing.
4. For a mail, multiply the prior with the conditional probabilities of all words: P(class) × Π P(wordi | class).
5. Assign the mail to the class with the highest value.
6. Predict all training records and build the confusion matrix.
7. Calculate the accuracy and classify a new mail.

8. Stop.

Data Set:

Spam keyword data set (12 records; 1 = word present, 0 = absent):

Free	Offer	Winner	Class
1	1	1	Spam
1	1	0	Spam
1	0	1	Spam
0	1	1	Spam
1	0	0	Spam
0	0	1	Spam
0	0	0	Ham
0	1	0	Ham
0	0	0	Ham
1	0	0	Ham
0	0	0	Ham
0	1	0	Ham

Python Code:

```
import numpy as np

# Spam data: Free, Offer, Winner present? (1/0) -> Spam / Ham
X = np.array([[1,1,1],[1,1,0],[1,0,1],[0,1,1],[1,0,0],[0,0,1],
              [0,0,0],[0,1,0],[0,0,0],[1,0,0],[0,0,0],[0,1,0]])
y =
np.array(["Spam","Spam","Spam","Spam","Spam","Spam",
              "Ham","Ham","Ham","Ham","Ham","Ham"])
classes = np.unique(y)

# Training: class priors and conditional probabilities (Laplace smoothing)
prior, cond = {}, {}
for c in classes:
    Xc = X[y == c]
    prior[c] = len(Xc) / len(X)
    cond[c] = (Xc.sum(axis=0) + 1) / (len(Xc) + 2)    # P(word = 1 | class)
```

```
def predict(x):
    score = {}
    for c in classes:
        p = prior[c]
        for j, v in enumerate(x):
            p *= cond[c][j] if v == 1 else 1 - cond[c][j]
        score[c] = p
    return max(score, key=score.get), score

pred = np.array([predict(x)[0] for x in X])
print("Prior probabilities:", {str(c): round(float(prior[c]), 2)
for c in classes})

tp = np.sum((pred == "Spam") & (y == "Spam")); fn =
np.sum((pred == "Ham") & (y == "Spam"))
fp = np.sum((pred == "Spam") & (y == "Ham"));  tn =
np.sum((pred == "Ham") & (y == "Ham"))
print("\nConfusion matrix (rows = actual Spam, Ham):")
print([[int(tp), int(fn)], [int(fp), int(tn)]])
print("Accuracy = %.2f%%" % (100 * np.mean(pred ==
y)))
```

```
label, sc = predict([1, 0, 1])
print("\nNew mail [Free=1, Offer=0, Winner=1] ->", label)
```

Output:

```
Prior probabilities: {'Ham': 0.5, 'Spam': 0.5}


Confusion matrix (rows = actual Spam, Ham):
[[5, 1], [0, 6]]
Accuracy = 91.67%


New mail [Free=1, Offer=0, Winner=1] -> Spam
```

Result:

The Naive Bayes classifier gave an accuracy of 91.67% with 11 of 12 records classified correctly. The new mail with the words Free and Winner was classified as Spam.

Question 2: Gaussian Naive Bayes on the Iris Data Set

Aim:

To implement the Gaussian Naive Bayes classifier on the Iris data set and to display the results using a confusion matrix and accuracy.

Algorithm:

1. Start. Import the libraries and load the Iris data set.
2. Split the data into training (70%) and testing (30%) sets.
3. Create a Gaussian Naive Bayes model, which assumes that each feature follows a normal distribution.
4. Train the model by finding the mean and variance of every feature for each class.
5. Predict the class of every test sample using Bayes' theorem.
6. Build the confusion matrix.
7. Calculate the accuracy and classify a new sample.
8. Stop.

Data Set:

Iris data set (150 records, 10 sample records shown):

Sepal Length	Sepal Width	Petal Length	Petal Width	Class
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.9	3.1	4.9	1.5	versicolor
6.3	3.3	6	2.5	virginica
5.8	2.7	5.1	1.9	virginica
7.1	3	5.9	2.1	virginica
5.9	3	5.1	1.8	virginica

Python Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score,
confusion_matrix

iris = load_iris()
Xtr, Xte, ytr, yte = train_test_split(iris.data, iris.target,
test_size=0.3,
                                      random_state=42,
stratify=iris.target)
nb = GaussianNB().fit(Xtr, ytr)
pred = nb.predict(Xte)

print("Confusion Matrix:")
print(confusion_matrix(yte, pred))
print("\nAccuracy = %.2f%%" % (100 *
accuracy_score(yte, pred)))

new = [[5.9, 3.0, 5.1, 1.8]]
print("New sample", new[0], "->",
iris.target_names[nb.predict(new)[0]])
```

Output:

Confusion Matrix:

[[15 0 0]

 [0 14 1]

 [0 3 12]]

Accuracy = 91.11%

New sample [5.9, 3.0, 5.1, 1.8] -> virginica

Result:

The Gaussian Naive Bayes model achieved an accuracy of 91.11%, with 41 of 45 test samples classified correctly. The new sample [5.9, 3.0, 5.1, 1.8] was classified as virginica.

Question 3: Naive Bayes Classification of the Breast Cancer Data Set

Aim:

To classify tumours as malignant or benign using the Naive Bayes algorithm and to evaluate it using the confusion matrix, accuracy, precision and recall.

Algorithm:

1. Start. Load the Breast Cancer data set and split it into training (75%) and testing (25%) sets.
2. Create the Gaussian Naive Bayes model.
3. Fit the model on the training data.
4. Predict the class labels of the test data.
5. Build the confusion matrix.
6. Calculate the accuracy.
7. Calculate the precision and recall and display the results.
8. Stop.

Data Set:

Breast Cancer data set (569 records, 30 features; first 8 records with 4 features shown):

Mean Radius	Mean Texture	Mean Perimeter	Mean Area	Class
17.99	10.38	122.8	1001	malignant
20.57	17.77	132.9	1326	malignant
19.69	21.25	130	1203	malignant
11.42	20.38	77.58	386.1	malignant
20.29	14.34	135.1	1297	malignant
12.45	15.7	82.57	477.1	malignant
18.25	19.98	119.6	1040	malignant
13.71	20.83	90.2	577.9	malignant

Python Code:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score,
confusion_matrix, precision_score, recall_score

data = load_breast_cancer()
Xtr, Xte, ytr, yte = train_test_split(data.data, data.target,
test_size=0.25,
                                      random_state=1,
stratify=data.target)
nb = GaussianNB().fit(Xtr, ytr)
pred = nb.predict(Xte)

print("Confusion Matrix:")
print(confusion_matrix(yte, pred))
print("\nAccuracy  = %.2f%%" % (100 *
accuracy_score(yte, pred)))
print("Precision = %.2f" % precision_score(yte, pred))
print("Recall    = %.2f" % recall_score(yte, pred))
```

Output:

```
Confusion Matrix:
[[48  5]
```

[5 85]]

Accuracy = 93.01%

Precision = 0.94

Recall   = 0.94

Result:

The Naive Bayes model achieved an accuracy of 93.01%, with 133 of 143 test samples classified correctly. The precision and recall were both 0.94.

Question 4: Multinomial Naive Bayes for Text Message Classification

Aim:

To classify text messages as spam or ham using the Multinomial Naive Bayes algorithm and to display the results using a confusion matrix and accuracy.

Algorithm:

1. Start. Load the messages and their labels (spam / ham).
2. Convert the text into a word-count matrix using CountVectorizer.
3. Create the Multinomial Naive Bayes model.
4. Train the model using the word counts and labels.
5. Predict the labels of the messages.
6. Build the confusion matrix and calculate the accuracy.
7. Convert new messages into word counts and classify them.
8. Stop.

Data Set:

Message data set (12 records):

Message	Class
win a free prize now	spam
free offer click now	spam
claim your free reward	spam
urgent winner claim cash prize	spam
lowest price offer buy now	spam
meeting at ten tomorrow	ham
please send the project report	ham
lunch with the team today	ham
call me when you are free	ham
notes for the exam tomorrow	ham
happy birthday see you today	ham
submit the lab record tomorrow	ham

Python Code:

```
from sklearn.feature_extraction.text import
CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score,
confusion_matrix

msgs = ["win a free prize now", "free offer click now",
"claim your free reward",
    "urgent winner claim cash prize", "lowest price offer
buy now",
    "meeting at ten tomorrow", "please send the project
report",
    "lunch with the team today", "call me when you are
free",
    "notes for the exam tomorrow", "happy birthday see
you today",
    "submit the lab record tomorrow"]
labels = ["spam"] * 5 + ["ham"] * 7

cv = CountVectorizer()
```

```
X = cv.fit_transform(msgs)
nb = MultinomialNB().fit(X, labels)
pred = nb.predict(X)

print("Confusion Matrix (labels: ham, spam):")
print(confusion_matrix(labels, pred, labels=["ham",
"spam"]))
print("Accuracy = %.2f%%" % (100 *
accuracy_score(labels, pred)))

for m in ["claim your free prize now", "project meeting
tomorrow"]:
    print(f'"{m}" ->', nb.predict(cv.transform([m]))[0])
```

Output:

```
Confusion Matrix (labels: ham, spam):
[[7 0]
 [0 5]]
Accuracy = 100.00%
"claim your free prize now" -> spam
"project meeting tomorrow" -> ham
```

Result:

The Multinomial Naive Bayes model classified all 12 messages correctly (accuracy 100%). The message "claim your free prize now" was classified as spam and "project meeting tomorrow" as ham.

Question 5: Gaussian Naive Bayes from Scratch for Gender Prediction

Aim:

To implement the Gaussian Naive Bayes algorithm from scratch using the mean, variance and Gaussian probability density function, and to predict the gender from height and weight.

Algorithm:

1. Start. Load the height and weight data with the gender labels.
2. Calculate the prior probability of each class.
3. Calculate the mean and variance of each feature for each class.
4. Define the Gaussian probability density: $P(x \mid class) = (1 / \sqrt{(2\pi\sigma^2)}) \times e^{(-(x - \mu)^2 / 2\sigma^2)}$.
5. For a sample, multiply the prior with the Gaussian probabilities of all features for each class.
6. Assign the sample to the class with the largest value.

7. Build the confusion matrix, find the accuracy and predict a new person.
8. Stop.

Data Set:

Height and weight data set (12 records):

Height (cm)	Weight (kg)	Gender
180	80	Male
175	76	Male
182	85	Male
178	82	Male
185	90	Male
172	74	Male
160	55	Female
165	58	Female
158	52	Female
162	56	Female
155	50	Female

Height (cm)	Weight (kg)	Gender
168	60	Female

Python Code:

```
import numpy as np

# Height (cm), Weight (kg) -> Gender
X = np.array([[180, 80], [175, 76], [182, 85], [178, 82],
[185, 90], [172, 74],
              [160, 55], [165, 58], [158, 52], [162, 56], [155, 50],
[168, 60]], float)
y = np.array(["Male"] * 6 + ["Female"] * 6)
classes = np.unique(y)

# Training: mean, variance and prior of every feature for
each class
stats = {c: (X[y == c].mean(axis=0), X[y == c].var(axis=0),
np.mean(y == c))
         for c in classes}
```

```
def gaussian(x, mean, var):
    return np.exp(-(x - mean) ** 2 / (2 * var)) / np.sqrt(2 *
np.pi * var)

def predict(x):
    score = {c: stats[c][2] * np.prod(gaussian(x, stats[c][0],
stats[c][1]))
             for c in classes}
    return max(score, key=score.get)

pred = np.array([predict(x) for x in X])
tp = np.sum((pred == "Male") & (y == "Male"));   fn =
np.sum((pred == "Female") & (y == "Male"))
fp = np.sum((pred == "Male") & (y == "Female")); tn =
np.sum((pred == "Female") & (y == "Female"))
print("Confusion matrix (rows = actual Male, Female):")
print([[int(tp), int(fn)], [int(fp), int(tn)]])
print("Accuracy = %.2f%%" % (100 * np.mean(pred ==
y)))

new = np.array([170, 66], float)
```

```
print("New person (170 cm, 66 kg) ->", predict(new))
```

Output:

```
Confusion matrix (rows = actual Male, Female):
[[6, 0], [0, 6]]
Accuracy = 100.00%
New person (170 cm, 66 kg) -> Male
```

Result:

The classifier predicted all 12 training records correctly (accuracy 100%). The new person (170 cm, 66 kg) was classified as Male.